# Latent World Models as Robot Foundation Models*

MD SELIM SAROWAR
selim.sarowar12@gmail.com | https://s-elim.github.io

**ABSTRACT** Whether a pretrained predictive model of dynamics is a transferable substrate for robot control has so far been argued from incomparable protocols. This report tests it under one protocol. V-JEPA 2-AC, DINO-WM, TD-MPC2 and behaviour cloning are trained on the same 160 human demonstrations of four LIBERO-Goal tasks (reach-and-press, pick-and-place, non-prehensile push and drawer opening) and evaluated on the same 40 held-out episodes with seed 0, through a shared 4 Hz, 7-DoF action interface that we verify keeps every task solvable. The two latent planners use frozen V-JEPA 2 ViT-L and DINOv2 features, 11M-parameter action-conditioned predictors and goal-image cross-entropy-method planning; TD-MPC2 is trained offline with LIBERO's sparse reward. Behaviour cloning succeeds in 52% of episodes (95% with a wrist camera), DINO-WM in 10%, V-JEPA 2-AC in 5% and TD-MPC2 in 0%, and every world-model success is on the reach-and-press task. The same frozen DINOv2 features give a policy 88% success and a planner 10%: the representation transfers, the dynamics learned from 40 demonstrations per task do not. Diagnostics locate the failure. The predictors beat a no-change baseline on demonstrated actions but are worse than it on the actions the planner selects, the planning energy tracks the arm rather than the manipulated object, imagined futures regress toward an average embedding, and most failures never reach the part to be manipulated. A three-step horizon raises V-JEPA 2-AC's reach-and-press success from 20% to 90% but leaves every contact-rich episode unsolved, and one planning decision costs 511 ms against 2 ms for the policy. Code, configurations, logs and per-episode records are released.

**INDEX TERMS** World models, joint-embedding predictive architecture, V-JEPA 2, DINO-WM, TD-MPC2, model predictive control, cross-entropy method, behaviour cloning, LIBERO, robot manipulation.

## I. INTRODUCTION

Assignment 1 reviewed twenty articles on learned world models used as pretrained foundation models for robot control and closed on one methodological gap: no article in that corpus trains a latent predictive model, a value-equivalent model and a direct policy on matched data and scores them on matched tasks [1, 4, 8, 15]. Every family reports on its own protocol, so the central question of the corpus, whether pretraining a predictive model of dynamics yields a transferable substrate for control, cannot be answered from published numbers. This report builds that comparison in simulation. Four models receive the same 160 LIBERO-Goal demonstrations [21], the same observation and action interface, the same simulator and the same 40 held-out evaluation episodes under one seed, and the analysis separates the three axes on which a world model can succeed or fail: the fidelity of its latent dynamics, its controllability (whether its predictions depend on the action), and the utility of both for closed-loop control.

### A. PROBLEM FORMULATION

Visual manipulation is a partially observed Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, \mathcal{R}, \gamma)$. In our instance the state $s \in \mathcal{S}$ is the full MuJoCo state of the LIBERO kitchen scene (79 numbers: simulation time and the positions and velocities of the robot joints, gripper fingers, four free objects, three drawer joints and the stove button). The robot never sees $s$. It observes $o_t = (I_t, q_t) \in \mathcal{O}$, a $128 \times 128$ RGB frame $I_t$ from the agent-view camera and a 7-D proprioceptive vector $q_t$ (end-effector position, axis-angle orientation, finger opening), produced by the observation kernel $\Omega(o \mid s)$, which here is the renderer plus forward kinematics. Actions $a_t \in \mathcal{A} = [-1, 1]^7$ are LIBERO's operational-space commands: the first three entries scale to an end-effector position offset of at most 0.05 m per control step, the next three to an orientation offset of at most 0.5 rad, and the last opens or closes the gripper. The transition kernel $\mathcal{T}(s' \mid s, a)$ is MuJoCo physics under an impedance controller at 20 Hz. The reward is LIBERO's sparse task signal, $\mathcal{R}(s) = \mathbb{1}[s \models g_\tau]$, where $g_\tau$ is the BDDL goal predicate of task $\tau$ (for example `(Open wooden_cabinet_1_middle_region)`), and episodes terminate on success.

The agent is asked to reach the goal from a held-out initial state within $T_{\max}$ control steps,

$$\pi^\star = \arg\max_{\pi} \Pr_{\pi,\mathcal{T}}\left[\exists t \le T_{\max} : \; s_t \models g_\tau \;\middle|\; s_0,\; c\right], \tag{1}$$

where the task specification $c$ is a goal image $I_g$ for the two latent planners and a task index $\tau$ for TD-MPC2 and behaviour cloning. In LIBERO-Goal every task has a single goal predicate in a single scene, so both specifications identify the same target; the goal image also fixes the final arm pose.

*Course code and repository: https://github.com/s-elim/AML_Assignment_World-Model. The repository holds the source code, configuration files, training and evaluation logs, per-episode results and the commands that reproduce every number, table and figure in this report.

## B. THE RECONSTRUCTION DILEMMA

A world model learns a predictive map $\hat{o}_{t+1} = F(o_{\leq t}, a_t)$ or, in latent form, $\hat{z}_{t+1} = f(z_{\leq t}, a_t)$ with $z = s_\phi(o)$. The two choices fail differently. Predicting pixels forces the model to spend capacity in proportion to pixel variance: a single $128 \times 128$ frame has $49{,}152$ values, most of which describe the textured kitchen rather than the 4 cm stove button or the drawer handle, and a decoder must be evaluated for every imagined frame, which makes sampling-based planning expensive [1, 12]. Predicting in a latent space removes the decoder and lets planning compare embeddings directly [8, 15], at three costs. First, an encoder trained jointly with its predictor can reach zero loss by mapping every observation to a constant, so latent methods need a stop-gradient, an exponential-moving-average target or a frozen encoder to prevent representation collapse [24, 25]. Second, a frozen encoder fixes what information survives: if the pretrained features do not resolve the state of a small articulated part, no predictor trained on top can recover it. Third, latent rollouts are not invertible, so an imagined plan cannot be inspected or checked against physics except through the planner's own energy. Value-equivalent models such as TD-MPC2 [4] take a third route, grounding the latent in reward and value instead of observations, which makes them task-specific and dependent on a reward signal.

## C. RESEARCH QUESTIONS

The study asks four questions, each tied to a measurement in Sections V to VII.

**RQ1.** *Control.* With 40 demonstrations per task, does goal-image planning in the frozen feature space of a self-supervised video encoder (V-JEPA 2-AC) solve LIBERO-Goal tasks, and how does it compare with planning in image features (DINO-WM), with a reward-grounded latent model (TD-MPC2) and with a direct policy (behaviour cloning) trained on the same data?

**RQ2.** *Fidelity and controllability.* How fast does open-loop latent prediction error grow with the horizon, does lower error translate into higher success, and do the models actually condition their predictions on the action?

**RQ3.** *Cost.* What latency, throughput and memory does test-time trajectory optimisation cost relative to a policy that maps observations to actions in one forward pass?

**RQ4.** *Design sensitivity.* How do planning horizon, sample budget, elite fraction and the latent distance change planning outcomes, and does freezing a pretrained representation help or hurt a policy trained directly on actions?

## D. CONTRIBUTIONS

1. *A matched protocol for the comparison Assignment 1 found missing.* Four model families from the Assignment 1 corpus and its baselines (V-JEPA 2-AC, DINO-WM, TD-MPC2, behaviour cloning) are trained on the same 160 LIBERO-Goal demonstrations and evaluated on the same 40 held-out episodes with paired statistics, after checking that the shared 4 Hz, 7-D action interface leaves every task solvable (Section IV-B).
2. *Faithful reduced-scale implementations.* V-JEPA 2-AC uses the released encoder and predictor code with an exact key/value-cached rollout that makes planning tractable; DINO-WM and TD-MPC2 follow their released configurations, and every deviation is listed (Section III).
3. *A diagnosis of where latent planning fails.* Beyond success rates, the study measures what the planning energy tracks, where imagined futures lie relative to real ones, how the energy surface compares with executed outcomes, and how failures divide between approach, contact, goal ambiguity and hallucination (Sections V-D and VI).
4. *Ablations that separate representation from dynamics.* Planner settings, a data-informed action prior, a post-hoc short horizon, and frozen versus fine-tuned encoders under a direct policy isolate which component limits control (Section VII).

**KEY FINDINGS**

- **RQ1, control.** On the 40 matched held-out episodes, behaviour cloning succeeds in 52% (and up to 95% with a wrist camera), DINO-WM in 10%, V-JEPA 2-AC in 5% and TD-MPC2 in 0%. Every world-model success is on the reach-and-press task.
- **The representation transfers, the dynamics do not.** The same frozen DINOv2 tokens give a policy 88% success and a planner 10%.
- **RQ2, fidelity and controllability.** On demonstrated actions both latent predictors beat a no-change baseline at every horizon (error ratio 0.93 to 0.74 for V-JEPA 2-AC, 0.77 to 0.57 for DINO-WM) and depend on their actions (shuffled-action error $\times 1.07$ to $\times 1.28$ and $\times 1.25$ to $\times 1.83$), yet on the actions the planner selects they are 25% to 36% worse than no change.
- **Mechanism.** The planning energy follows the arm ($\rho_s$ 0.36 and 0.52) rather than the object (0.21 and 0.10), imagined futures regress toward an average embedding, and most planner failures never reach the part to be manipulated.
- **RQ3, cost.** A V-JEPA 2-AC decision takes 511 ms against 2 ms for the policy.
- **RQ4, design.** Only the horizon matters among planner settings: three-step plans raise V-JEPA 2-AC's Stove success from 2/10 to 9/10 ($p = 0.039$). Re-run on all tasks after this choice, the three-step planners still solve none of the 60 contact-rich episodes (9/40 and 5/40 pooled).

## II. THEORETICAL FOUNDATIONS AND CORPUS TAXONOMY

The twenty articles of Assignment 1 fall into families that differ in what they predict, in which space, and how prediction becomes action. This section writes the objective of each family in one notation, because the objective determines both what a model can represent and how it can be used for control. Throughout, $h$ denotes a deterministic recurrent or latent state, $z$ a stochastic or encoded latent, and $\mathrm{sg}(\cdot)$ the stop-gradient operator.

### A. RECURRENT STATE-SPACE MODELS WITH PIXEL RECONSTRUCTION

DayDreamer [1] and DreamerV3 [12] train a recurrent state-space model (RSSM) whose deterministic state follows $h_t = f(h_{t-1}, z_{t-1}, a_{t-1})$. A posterior $q(z_t \mid h_t, o_t)$ and a prior $p(z_t \mid h_t)$ over the stochastic state are fitted by maximising an evidence lower bound,

$$\mathcal{L}_{\mathrm{RSSM}} = \mathbb{E}\Big[\sum_t \underbrace{-\ln p(o_t \mid h_t, z_t)}_{\text{reconstruction}} - \ln p(r_t \mid h_t, z_t) + \beta\,\mathrm{KL}\big(q(z_t \mid h_t, o_t) \,\|\, p(z_t \mid h_t)\big)\Big]. \tag{2}$$

The reconstruction term prevents collapse because a constant latent cannot reproduce the image, but it also makes every bit of image entropy a training target. DreamerV3 stabilises Eq. (2) across domains with symlog targets, two-hot value regression and free bits [12]; the policy is then learned on imagined rollouts. iVideoGPT [7] replaces the RSSM by an autoregressive transformer over compressed visual tokens and keeps the reconstruction anchor.

### B. LATENT PREDICTION WITHOUT RECONSTRUCTION

DINO-WM [8], Navigation World Models [9] and V-JEPA 2 [15] delete the decoder. With an encoder $s_\phi$ and a predictor $f_\theta$,

$$\mathcal{L}_{\mathrm{JEPA}}(\theta) = \mathbb{E}\Big[\big\| f_\theta\big(s_\phi(o_{\le t}), a_{\le t}\big) - \mathrm{sg}\big(s_{\bar\phi}(o_{t+1})\big)\big\|_p^p\Big], \qquad p \in \{1, 2\}. \tag{3}$$

If $\phi$ and $\theta$ were trained jointly without further constraint, $s_\phi \equiv \text{const}$ would minimise Eq. (3). The families avoid this in two ways. V-JEPA and V-JEPA 2 pretraining use a target encoder $s_{\bar\phi}$ whose weights are an exponential moving average of $\phi$, a stop-gradient on the target, and a masked-prediction task that makes the constant solution unstable [24, 25]. DINO-WM and the action-conditioned stage of V-JEPA 2 freeze the encoder altogether ($\bar\phi = \phi$, no gradient), so collapse is impossible by construction and the predictor is the only trainable part. Navigation World Models predict future latents with a conditional diffusion transformer rather than a regression head [9].

### C. VALUE-EQUIVALENT LATENT DYNAMICS

TD-MPC2 [4] grounds its latent in the task instead of the observation. An encoder $z_t = h_\xi(o_t)$, latent dynamics $d_\chi$, reward head $r_\psi$, an ensemble of action-value heads $Q_{\nu_i}$ and a policy prior $\pi_\eta$ are trained jointly over a short horizon $H_\ell$,

$$\mathcal{L}_{\text{TD-MPC2}} = \sum_{t=0}^{H_\ell - 1} \rho^t \Big[c_d \big\| d_\chi(\hat z_t, a_t) - \mathrm{sg}(h_\xi(o_{t+1}))\big\|_2^2 + c_r\,\mathrm{CE}\big(\hat r_t, r_t\big) + c_q\,\mathrm{CE}\big(\hat q_t,\; r_t + \gamma\,\bar Q(z_{t+1}, \pi_\eta(z_{t+1}))\big)\Big], \tag{4}$$

with $\hat z_0 = h_\xi(o_0)$, $\hat z_{t+1} = d_\chi(\hat z_t, a_t)$, cross-entropy over a discrete two-hot encoding of reward and value, and $\bar Q$ a slowly updated target network. Collapse is limited because a constant latent cannot predict reward and value; the price is that the latent only needs to encode what affects return under the training reward.

### D. NON-PREDICTIVE IMITATION

Behaviour cloning fits $\pi_\eta(a \mid o)$ by maximum likelihood on demonstrations, $\mathcal{L}_{\mathrm{BC}} = -\mathbb{E}_{(o,a)\sim\mathcal{D}} \ln \pi_\eta(a \mid o)$, and uses no model of dynamics. It is the reference against which any claim that prediction helps control must be measured, which is why Assignment 1 recommended it as the baseline for every world-model evaluation [28].

### E. VIDEO GENERATION AND JOINT VIDEO-ACTION MODELS

The remaining families of the corpus generate pixels as a policy substrate (UniPi, UniSim, GR-1, VPP, DreamGen, Genie Envisioner [2, 3, 5, 10, 14, 16]) or model video and action jointly (UWM, Cosmos, Ctrl-World [13, 11, 17]), and the 2026 evaluation papers audit them [18, 19, 20]. These systems require billion-parameter video generators and are outside the compute of this study; Table 1 lists them for completeness and the four evaluated families are marked.

## III. MODEL ARCHITECTURES AND ALGORITHMS

Fig. 1 summarises the four pipelines. All of them act at the same decision rate: one *macro step* spans $k = 5$ control steps (0.25 s), the rate of the 4 frames per second used to train V-JEPA 2-AC [15]. The macro action $\bar a_t = \frac{1}{k}\sum_{j=0}^{k-1} a_{t+j}$ is the mean of the demonstrated commands it spans and is executed under a zero-order hold. Section IV-B verifies that this interface preserves task feasibility.

**TABLE 1.** Taxonomy of the Assignment 1 corpus by training objective. $N$ is the number of tokens per frame, $d$ the model width, $L$ the depth, $HW$ the pixel count and $T$ the number of frames in context. Families marked • are evaluated in this study.

| Family | Corpus | Objective (what is predicted) | Collapse mitigation | Cost per predicted step |
|---|---|---|---|---|
| RSSM with reconstruction | [1, 12] | ELBO, Eq. (2) (pixels, reward) | decoder anchors $z$; KL with free bits | $O(d^2)$ + decoder $O(HW)$ |
| Token video model | [7, 6] | next visual token (cross-entropy) | reconstruction via tokenizer | $O(LTN\,d^2)$ |
| • Latent, no decoder | [8, 9, 15] | Eq. (3) (frozen or EMA features) | frozen encoder or EMA target + sg | $O(L(Nd^2{+}TN^2d))$ |
| • Value-equivalent | [4] | Eq. (4) (latent, reward, value) | reward and value heads; SimNorm | $O(d^2)$ |
| Video as policy | [2, 3, 5, 10, 14, 16] | diffusion denoising of frames | reconstruction | diffusion steps × $O(LTN\,d^2)$ |
| Joint video-action | [13, 11, 17] | joint diffusion over frames and actions | reconstruction | as above |
| • Direct imitation | baseline [28] | $\ln \pi(a \mid o)$ | not applicable | one encoder pass |
| Evaluation studies | [18, 19, 20] | none (audit protocols) | not applicable | not applicable |

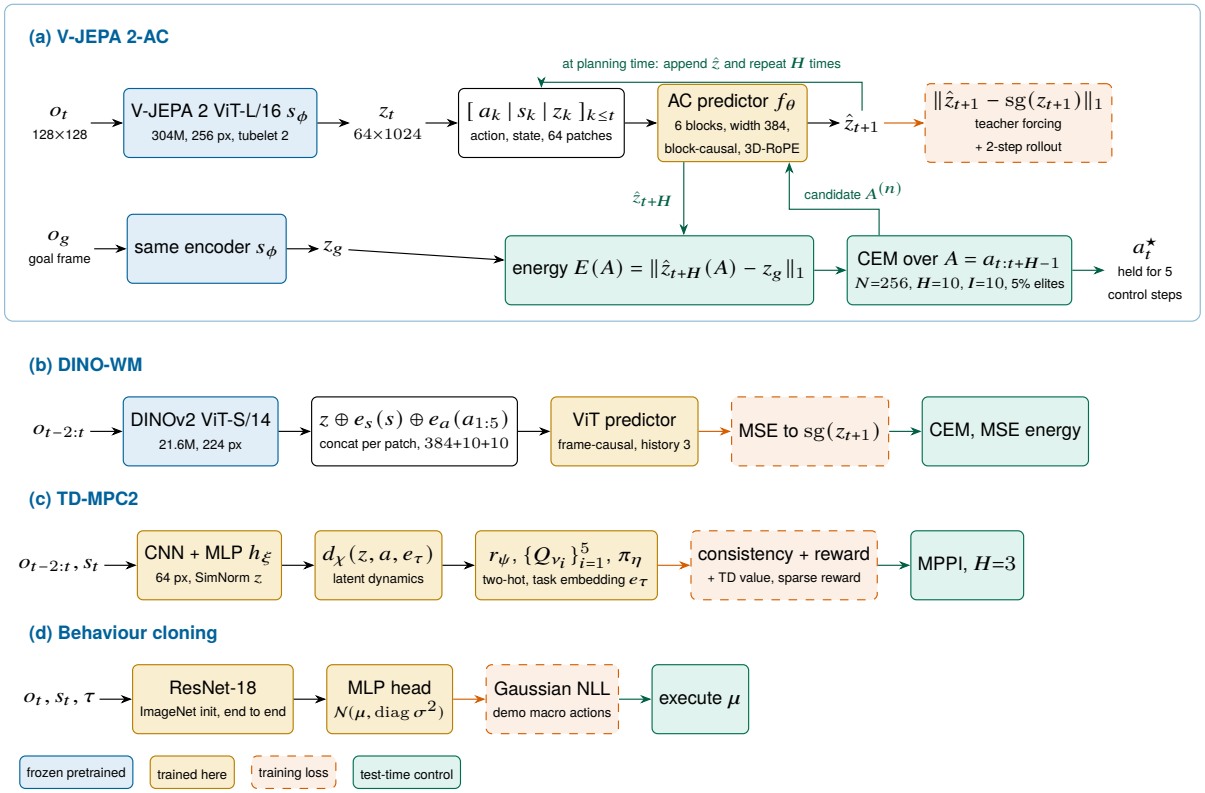



**FIGURE 1.** The four evaluated pipelines. (a) V-JEPA 2-AC: frozen V-JEPA 2 features, an action-conditioned block-causal predictor trained with the release's L1 teacher-forcing and rollout losses, and test-time CEM toward a goal embedding. (b) DINO-WM: frozen DINOv2 features with action and proprioception embeddings concatenated per patch. (c) TD-MPC2: a task-grounded latent trained with consistency, reward and TD losses, controlled by MPPI. (d) Behaviour cloning. Blue blocks are frozen pretrained networks, orange blocks are trained on the 160 demonstrations.

## A. ACTION-CONDITIONED V-JEPA 2 (V-JEPA 2-AC)

*1) Encoder.* The encoder $s_\phi$ is the released V-JEPA 2 ViT-L/16 [37] target encoder (304M parameters), pretrained by masked latent prediction on more than one million hours of video [15]. Following the V-JEPA 2-AC training code, a single frame is upsampled to the $256\times256$ pretraining resolution and duplicated into one two-frame tubelet, which yields $16\times16$ patch tokens of width 1024. We average-pool them to an $8\times8$ grid and normalise each token,

$$z_t = \mathrm{LN}\big(\mathrm{Pool}_{2\times2}\, s_\phi([I_t, I_t])\big) \in \mathbb{R}^{P\times D}, \qquad P = 64,\ D = 1024, \tag{5}$$

where LN is layer normalisation without affine parameters (the release's `normalize_reps`). The encoder is never updated, so the target of Eq. (8) carries no gradient and collapse cannot occur.

*2) Action conditioning.* V-JEPA 2-AC is sometimes described as conditioning on actions through cross-attention, but the released predictor conditions by token interleaving: for every frame $t$ the sequence contains one action token, one proprioceptive-state token and the 64 patch tokens,

$$X_t = \big[\, W_a \bar{a}_t,\ W_s \tilde{q}_t,\ W_z z_t^{(1)}, \ldots, W_z z_t^{(P)} \,\big] \in \mathbb{R}^{(P+2)\times d}, \tag{6}$$

where $g_\psi = (W_a, W_s, W_z)$ are linear embeddings into the predictor width $d = 384$ and $\tilde{q}_t$ is the standardised 7-D state. Action information reaches the patch tokens through self-attention.

*3) Predictor.* The predictor $f_\theta$ is the released `VisionTransformerPredictorAC`, vendored unchanged and instantiated with 6 blocks of width 384 and 6 heads (11.4M parameters; the release uses 24 blocks of width 1024 on ViT-g features). Attention is block-causal over frames: with $\mathrm{fr}(i)$ the frame of token $i$, the mask is $M_{ij} = \mathbb{1}[\mathrm{fr}(i) \geq \mathrm{fr}(j)]$, so every token sees its own frame and all earlier ones. Positions use three-dimensional rotary embeddings [38] over (time, row, column), with the action and state tokens rotated by time only. The output at the patch positions of frame $t$ is the prediction of frame $t+1$,

$$\hat{z}_{t+1} = \mathrm{LN}\Big(W_o\, f_\theta\big(X_0, \ldots, X_t; M\big)_t\Big). \tag{7}$$

*4) Training objective.* Clips of $T = 8$ macro frames (2 s) are encoded once. The loss is the release's sum of an L1 teacher-forcing term over the whole clip and an L1 term on a two-step autoregressive rollout in which the predictor consumes its own first prediction,

$$\mathcal{L}_{\mathrm{AC}} = \underbrace{\frac{1}{T-1}\sum_{t=0}^{T-2}\frac{\|\hat{z}_{t+1} - \mathrm{sg}(z_{t+1})\|_1}{PD}}_{\text{teacher forcing}} + \underbrace{\frac{1}{2}\sum_{n=1}^{2}\frac{\|\tilde{z}_n - \mathrm{sg}(z_n)\|_1}{PD}}_{\text{rollout, } \tilde{z}_1 = \hat{z}_1,\ \tilde{z}_2 = f_\theta(z_0, \tilde{z}_1)}. \tag{8}$$

*5) Planning-time state and cached rollout.* The release plans with measured pose deltas, so it advances the state token exactly by composing poses. Our actions are controller commands, so we advance the state with a linear map fitted by least squares on the training transitions, $\tilde{q}_{t+1} = \tilde{q}_t + W_q[\tilde{q}_t; \bar{a}_t; 1]$, which explains $R^2 = 0.71$ of the state change on the training set. Because attention is block-causal, the keys and values of frames $0, \ldots, t$ do not change when frame $t+1$ is appended. Caching them makes an $H$-step rollout process $H(P+2)$ new tokens instead of $\frac{1}{2}H(H+1)(P+2)$ (5.5 times fewer at $H = 10$) and is exact: our implementation matches the release's full re-forward to $10^{-4}$ (`tests/test_vjepa2_ac.py`).

## B. DINO-WM

*1) Frozen ViT tokens and registers.* DINO-WM encodes frames with a frozen DINOv2 ViT-S/14 [22] (21.6M parameters). Frames are upsampled to $224 \times 224$, giving $16 \times 16$ patch tokens of width 384 that we pool to $8 \times 8$ and normalise as in Eq. (5), so both latent planners see the same token budget. The checkpoint used by DINO-WM, `dinov2_vits14`, has no register tokens [23]; the class token is discarded and only patch tokens enter the model.

*2) Transition model.* Proprioception and the $k$ primitive actions of a macro step are embedded to 10 dimensions each and concatenated to every patch token along the feature axis,

$$x_t^{(p)} = \big[z_t^{(p)};\, e_q(\tilde{q}_t);\, e_a(a_{t:t+k-1})\big] \in \mathbb{R}^{404}, \tag{9}$$

and a ViT predictor $T_\omega$ with learned positional embeddings and a frame-causal mask over a history of three frames predicts the next frame's tokens, $\hat{x}_{t+1} = T_\omega(x_{t-2:t})$. We keep DINO-WM's depth of 6 and reduce the width to 6 heads of size 64 and an MLP of 1536 (11.3M parameters, matched to the V-JEPA 2-AC predictor).

*3) Objective.* Teacher-forced mean squared error on the visual and proprioceptive parts of the next embedding, with stop-gradient targets:

$$\mathcal{L}_{\text{DINO-WM}} = \big\|\hat{x}^{\mathrm{vis}}_{t+1} - \mathrm{sg}(z_{t+1})\big\|_2^2/PD + \big\|\hat{x}^{\mathrm{prop}}_{t+1} - \mathrm{sg}(e_q(\tilde{q}_{t+1}))\big\|_2^2/10. \tag{10}$$

At planning time the predicted proprioceptive embedding is fed back and the action slot is overwritten with the candidate action, as in the release.

## C. TD-MPC2

We re-implement the 5M-parameter configuration of TD-MPC2 [4]. The encoder $h_\xi$ is the paper's four-layer convolutional network on a stack of three $64 \times 64$ frames, fused with a proprioception MLP into a 512-D latent normalised by SimNorm (softmax over groups of 8). A learnable task embedding $e_\tau \in \mathbb{R}^{96}$ is concatenated to the inputs of the dynamics $d_\chi(z, a, e_\tau)$, the reward head $r_\psi$, five action-value heads $Q_{\nu_i}$ (1% dropout, Polyak targets with $\tau = 0.01$) and the policy prior $\pi_\eta$, all LayerNorm-Mish MLPs of width 512. Reward and value are regressed as two-hot distributions over 101 bins in symlog space. The loss is Eq. (4) with $(c_d, c_r, c_q) = (20, 0.1, 0.1)$, $\rho = 0.5$ and $H_\ell = 3$; the policy maximises $Q$ normalised by a running 5th-to-95th percentile scale plus an entropy bonus of $10^{-4}$. The paper trains online; here TD-MPC2 trains offline on the same 160 demonstrations with LIBERO's sparse reward ($r = 1$ on the macro step that reaches the goal, termination there), and the discount follows the paper's heuristic for a 60-step episode, $\gamma = 0.95$. Training applies random-shift augmentation [39] of $\pm 4$ pixels. Control uses MPPI with 512 samples, 64 elites, 24 policy-prior trajectories, 6 iterations, horizon 3 and temperature 0.5, warm-started from the previous plan [33, 4].

## D. BEHAVIOUR CLONING

The policy maps the agent-view frame (ResNet-18 [36], ImageNet initialisation, trained end to end), the standardised state and a 32-D task embedding through a two-layer MLP to a diagonal Gaussian over the 7-D macro action, and is trained by negative

log-likelihood,

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}\Big[\sum_{j=1}^{7} \frac{(\bar{a}_j - \mu_j)^2}{2\sigma_j^2} + \ln \sigma_j\Big] + \text{const}, \qquad (\mu, \ln \sigma) = \pi_\eta(I_t, \tilde{q}_t, \tau). \tag{11}$$

The executed action is $\mu$. Four variants form the representation ablation of Section VII: a second ResNet-18 on the wrist camera, attention pooling over frozen DINOv2 or frozen V-JEPA 2 tokens, and DINOv2 fine-tuned end to end.

### E. LATENT MODEL PREDICTIVE CONTROL WITH THE CROSS-ENTROPY METHOD

Both latent world models plan by minimising an energy between the imagined final frame and the goal embedding $z_g = s_\phi(I_g)$,

$$A^\star = \arg\min_{A=(\bar{a}_t,\ldots,\bar{a}_{t+H-1})\in[-1,1]^{H\times 7}} E(A), \qquad E(A) = D\big(\hat{z}_{t+H}(A),\, z_g\big), \tag{12}$$

with the cross-entropy method [31], the standard optimiser of visual model-predictive control from pixel-space Visual Foresight [35] to latent-space PlaNet [34], in the form of the released V-JEPA 2-AC planner (Algorithm 1, Fig. 2): a diagonal Gaussian per step, top-$K$ elites, momentum on mean and standard deviation, and execution of the first mean action before replanning. Unlike the release, which plans translation and gripper only, we optimise all seven dimensions, because Section IV-B shows that two of the four tasks cannot be completed without rotation. The default distances are the planning energies of the two papers,

$$D_{\mathrm{L1}}(\hat{z}, z_g) = \tfrac{1}{PD}\|\hat{z} - z_g\|_1 \text{ (V-JEPA 2-AC)}, \qquad D_{\mathrm{MSE}}(\hat{z}, z_g) = \tfrac{1}{PD}\|\hat{z} - z_g\|_2^2 \text{ (DINO-WM)}, \tag{13}$$

and the ablation of Section VII adds the Euclidean distance $D_{\mathrm{L2}} = \|\hat{z} - z_g\|_2$, the cosine distance $D_{\cos} = 1 - \langle \hat{z}, z_g\rangle / (\|\hat{z}\|\|z_g\|)$ on the flattened grids, and a diagonal Mahalanobis distance $D_{\mathrm{M}} = \big(\frac{1}{PD}\sum_{p,c}(\hat{z}_{pc} - z_{g,pc})^2/\sigma_c^2\big)^{1/2}$ with per-channel variances $\sigma_c^2$ estimated on training tokens.

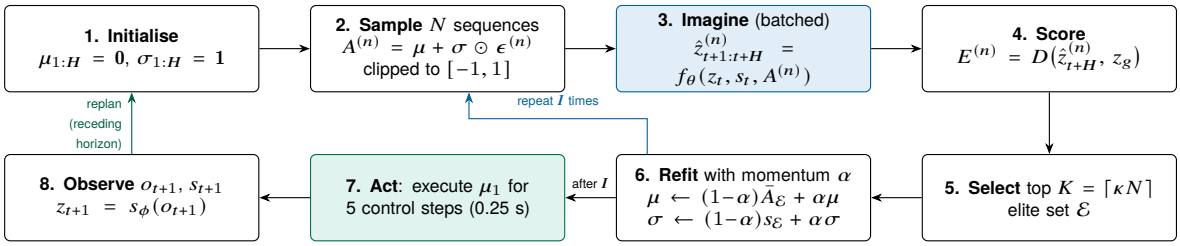



**FIGURE 2.** One decision of receding-horizon latent CEM. Steps 2 to 6 run $I$ times on the GPU; only the first action of the final mean is executed, after which the new observation is encoded and the optimisation restarts from a zero mean, as in the released V-JEPA 2-AC planner.

Defaults are $H = 10$ macro steps (2.5 s), $N = 256$, $I = 10$, $\kappa = 0.05$ ($K = 13$), $\alpha = 0.15$ and $\sigma_{\min} = 0.02$; the AR(1) smoothing of the sampling noise, in the spirit of colored-noise CEM [32], is available but disabled ($\beta = 0$). For reference, the V-JEPA 2 paper deploys $N = 800$, $K = 10$, $I = 10$ and a one-step horizon on a real Franka arm [15]; DINO-WM uses $N = 300$, $K = 30$ and 30 iterations [8].

## IV. EXPERIMENTAL SETUP

### A. SIMULATOR AND TASKS

All experiments run in LIBERO [21], a MuJoCo [27] and robosuite [26] benchmark whose tasks are specified as BDDL goal predicates. We wrap it behind the Gymnasium API (`src/env/libero_env.py`): `reset` loads a stored MuJoCo state, `step` executes one macro action for five control steps, and observations are the agent-view and wrist RGB frames at $128\times128$ plus the 7-D state. The four tasks come from the LIBERO-Goal suite, whose ten tasks share one kitchen scene and one Franka Panda, so that a single model can serve all of them and differences between tasks come from the skill rather than from the scene (Table 2). They span a short reach-and-press, a prehensile pick-and-place, a non-prehensile push and the opening of an articulated drawer.

### B. DATASET, PREPROCESSING AND SPLIT

The data are the 50 human teleoperated demonstrations per task released with LIBERO (200 in total), collected by human teleoperation through the same operational-space controller that our agents command. Three preprocessing steps were necessary and are documented because each changes what the models see.

*1) Re-rendering.* The released frames differ from a fresh render of the same MuJoCo state by 3.7 intensity levels on average (out of 255), because they were rendered with a different MuJoCo build. A model trained on released frames and evaluated on live frames would therefore face a small but systematic input shift. We regenerate every frame from the stored states with the evaluation renderer (`scripts/build_libero_dataset.py`); all 200 regenerated demonstrations end in a state that satisfies the BDDL goal.

**Algorithm 1** Receding-horizon latent CEM (one decision).

**Require:** frame $I_t$, state $q_t$, goal embedding $z_g$, predictor $f_\theta$, horizon $H$, samples $N$, iterations $I$, elite fraction $\kappa$, momentum $\alpha$, smoothing $\beta$, floor $\sigma_{\min}$

$z_t \leftarrow \mathrm{LN}(\mathrm{Pool}(s_\phi(I_t)))$; $\mu_{1:H} \leftarrow \mathbf{0}$; $\sigma_{1:H} \leftarrow \mathbf{1}$; $K \leftarrow \max(2, \mathrm{round}(\kappa N))$

**for** $i = 1, \ldots, I$ **do**

  draw $\epsilon_1^{(n)} \sim \mathcal{N}(0, I_7)$; for $h > 1$: $\epsilon_h^{(n)} \leftarrow \beta\, \epsilon_{h-1}^{(n)} + \sqrt{1-\beta^2}\, \xi_h^{(n)}$, $\xi \sim \mathcal{N}(0, I_7)$ ▷ temporal smoothing

  $A^{(n)} \leftarrow \mathrm{clip}_{[-1,1]}\left(\mu + \sigma \odot \epsilon^{(n)}\right)$ for $n = 1, \ldots, N$ ▷ action bounds

  $\hat{z}^{(n)}_{t+1:t+H} \leftarrow f_\theta(z_t, \tilde{q}_t, A^{(n)})$ in one batched, key/value-cached pass

  $E^{(n)} \leftarrow D(\hat{z}^{(n)}_{t+H}, z_g)$; $\mathcal{E} \leftarrow$ indices of the $K$ smallest $E^{(n)}$ ▷ elite selection

  $\mu \leftarrow (1-\alpha)\, \mathrm{mean}_{n\in\mathcal{E}} A^{(n)} + \alpha\, \mu$; $\sigma \leftarrow \max\left(\sigma_{\min}, (1-\alpha)\, \mathrm{std}_{n\in\mathcal{E}} A^{(n)} + \alpha\, \sigma\right)$ ▷ Gaussian refit

**return** $\mu_1$ (executed for $k = 5$ control steps, then replan)

**TABLE 2.** The four LIBERO-Goal tasks. Lengths are in 20 Hz control steps over the 50 demonstrations (mean ± standard deviation). The goal error, reported in centimetres in Section V, measures the task-relevant part against the final state of the held-out demonstration.

| Short name | LIBERO instruction | Skill | BDDL goal | Length | Goal error |
|---|---|---|---|---|---|
| Stove | turn on the stove | reach and press | `(Turnon flat_stove_1)` | 90.2 ± 7.9 | gripper position |
| Bowl-Plate | put the bowl on the plate | pick and place | `(On akita_black_bowl_1 plate_1)` | 94.4 ± 9.2 | bowl position |
| Push-Plate | push the plate to the front of the stove | non-prehensile push | `(On plate_1 main_table_stove_front_region)` | 153.8 ± 26.2 | plate position |
| Drawer | open the middle drawer of the cabinet | articulated | `(Open wooden_cabinet_1_middle_region)` | 141.5 ± 18.2 | drawer-joint travel |

*2) Alignment.* In the LIBERO files `obs[i]` is recorded after `actions[i]` is executed, so the common pairing (`obs[i]`, `actions[i]`) is shifted by one step. Our regenerated data render observation $t$ at the state that precedes action $t$, which we verified against the stored proprioception (maximum deviation 0.006).

*3) Split and macro steps.* Ten demonstrations per task are held out by a permutation drawn with seed 0 (indices 2, 4, 10, 11, 22, 27, 28, 31, 38 and 41, identical for every task), leaving 160 training demonstrations with 19,001 frames and 40 test demonstrations with 4,993 frames. There is no separate validation split because no checkpoint or hyperparameter is selected on held-out data: every configuration was fixed before evaluation, the held-out loss is logged for monitoring only, and the single post-hoc choice (the three-step horizon of Section VII-A5) is reported as such. Training clips use every offset of the macro grid, giving 13,401 eight-frame clips for V-JEPA 2-AC, 16,601 four-frame clips for DINO-WM and 18,841 labelled frames for behaviour cloning.

*4) Interface feasibility.* Before training anything we tested whether a policy restricted to our action interface can solve the tasks, by replaying ten demonstrations per task open loop (Table 3). Removing the rotation commands, as the released V-JEPA 2-AC planner does, makes Stove and Drawer unsolvable (0/10 each), so all models act in the full 7-D space. Averaging the commands over five control steps and holding them, the 4 Hz interface of every model in this study, solves 38 of 40 replays, the same count as replaying the original 20 Hz commands. The interface therefore leaves the tasks solvable. Both 7-D interfaces fail on two replays in the contact-rich Bowl-Plate and Push-Plate tasks, which we attribute to the absence of feedback in open-loop replay rather than to the interface.

**TABLE 3.** Open-loop replay of ten demonstrations per task under three action interfaces (successes out of 10).

| Interface | Stove | Bowl-Plate | Push-Plate | Drawer | Total |
|---|---|---|---|---|---|
| Original 20 Hz 7-D commands | 10 | 9 | 9 | 10 | 38/40 |
| 20 Hz, rotation removed (4-DoF translation + gripper) | 0 | 10 | 6 | 0 | 16/40 |
| **4 Hz macro steps, 7-D mean command held for 5 steps (ours)** | 10 | 10 | 8 | 10 | 38/40 |

## C. TRAINING CONFIGURATION

Every model is a single network for all four tasks, trained once with seed 0 and evaluated at its final checkpoint; the held-out loss is logged for monitoring only and no checkpoint is selected on it. Table 4 lists the configuration required by the assignment for each model. The two world models share optimiser settings (the V-JEPA 2-AC release's peak learning rate, warm-up and weight decay) so that the comparison between them isolates the representation and the predictor design.

## D. EVALUATION PROTOCOL AND METRICS

Each of the 40 held-out demonstrations defines one evaluation episode: the simulator is reset to the demonstration's first MuJoCo state and the goal frame is rendered from its final, successful state. This gives every agent the same initial conditions and every planner a goal image that is reachable from the start, and the four agents are evaluated on identical episodes, which makes their

**TABLE 4.** Training configuration. LR is the peak learning rate; "frozen" counts pretrained parameters that are not updated. Wall-clock times are for one NVIDIA RTX PRO 6000 Blackwell shared with the other runs of the stage.

| | V-JEPA 2-AC | DINO-WM | TD-MPC2 | BC (ResNet-18) |
|---|---|---|---|---|
| Input | view 256 px + state | view 224 px + state | 3 views 64 px + state + task | view 128 px + state + task |
| Pretrained | V-JEPA 2 ViT-L/16 | DINOv2 ViT-S/14 | none | ResNet-18 (ImageNet) |
| Frozen/trained | 304M / 11.4M | 21.6M / 11.3M | 0 / 5.6M | 0 / 11.8M |
| Loss | L1, Eq. (8) | MSE, Eq. (10) | Eq. (4) | Gaussian NLL, Eq. (11) |
| Optimiser | AdamW | AdamW | Adam | AdamW |
| LR / schedule | $4.25\times10^{-4}$, cosine | $4.25\times10^{-4}$, cosine | $3\times10^{-4}$, enc. $\times0.3$ | $10^{-4}$, one-cycle |
| Weight decay | 0.04 | 0.04 | 0 | $10^{-4}$ |
| Batch | 64 clips of 8 frames | 64 clips of 4 frames | 256 sequences of 4 | 64 frames |
| Epochs / updates | 40 epochs (8,360) | 40 epochs (10,360) | 40,000 updates | 30 epochs (8,820) |
| Augmentation | none (cached features) | none (cached features) | random shift ±4 px | random shift ±6 px |
| Output | next-frame tokens $64\times1024$; action via CEM (7-D) | next-frame tokens $64\times404$; action via CEM | next latent (512), reward, 5 values; MPPI | $\mathcal{N}$ over 7-D macro action |
| Precision | bf16 autocast | bf16 autocast | fp32 | bf16 autocast |
| Wall clock | 17 min | 5 min | 73 min | 19 min |

outcomes paired. An episode ends when the BDDL predicate holds or after 300 control steps (60 decisions), the LIBERO-Goal limit used by OpenVLA's evaluation [29]. All randomness (data split, initialisation, minibatch order, CEM and MPPI sampling) is seeded with 0, and the planner's generator is re-seeded at the start of every episode.

Safety events such as collisions are not instrumented in LIBERO and are not reported. We report the success rate with a Wilson 95% interval [42]; the final goal error in centimetres (Table 2); the return, which under LIBERO's sparse reward equals the success indicator and is reported once for completeness; the number of control steps to success; planning latency, throughput and peak GPU memory measured in isolation (Section V-C); and open-loop latent prediction error at horizons $k \in \{1, 3, 5, 10\}$ (Section V-B). Because outcomes are paired, differences in success between two agents are tested with the exact McNemar test [43] on the 40 matched episodes, and differences in final goal error with the Wilcoxon signed-rank test. With 10 episodes per task the per-task intervals are wide (a 5/10 result has a Wilson interval of 24% to 76%), so per-task differences are interpreted as indicative and pooled differences as the primary evidence.

### E. HARDWARE AND SOFTWARE

The compute cluster provides four NVIDIA RTX PRO 6000 Blackwell Server Edition GPUs (96 GB each) on two dual-socket Intel Xeon 6517P nodes. Training, feature encoding and planning for this report ran on one of these GPUs in a single Slurm job with 16 CPU cores, so all wall-clock times and latencies refer to one GPU; dataset regeneration and some tests ran on a 48-thread Intel Xeon Silver 4510 head node with software OSMesa rendering. Software: Python 3.11.16, PyTorch 2.11.0 with CUDA 12.8, timm 1.0.30, MuJoCo 2.3.7, robosuite 1.4.0, LIBERO (commit `8145de4`), NumPy 1.26.4 and Gymnasium 1.4.0. Rendering used EGL on the GPU node and OSMesa on the head node. The latent planners were evaluated on the GPU node; behaviour cloning, TD-MPC2 and the open-loop and energy analyses ran on the head-node CPU, which changes wall-clock time but not outcomes, and all latencies reported in Table 8 were measured separately on the GPU.

## V. QUANTITATIVE RESULTS AND COMPARATIVE ANALYSIS

### A. CLOSED-LOOP CONTROL (RQ1)

**TABLE 5.** Closed-loop results on the 40 held-out episodes (seed 0). Successes per task are out of 10; the pooled success rate is followed by its Wilson 95% interval. Goal error is the final distance of the task-relevant part from its goal configuration (Table 2); steps are 20 Hz control steps over successful episodes. Under LIBERO's sparse reward the return equals the success indicator, so the pooled success column is also the mean return.

| | Successes (of 10) | | | | Success % [95% CI] | Final goal error (cm) | | | | Mean | Steps to |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Model | Stove | Bowl | Push | Drawer | (pooled, $n$=40) | Stove | Bowl | Push | Drawer | error | success |
| V-JEPA 2-AC | 2/10 | 0/10 | 0/10 | 0/10 | **5** [1, 17] | 34.0 | 17.1 | 24.9 | 15.9 | 23.0 | 150 |
| DINO-WM | 4/10 | 0/10 | 0/10 | 0/10 | **10** [4, 23] | 20.3 | 17.4 | 26.1 | 15.9 | 19.9 | 160 |
| TD-MPC2 | 0/10 | 0/10 | 0/10 | 0/10 | **0** [0, 9] | 72.6 | 14.0 | 24.9 | 15.8 | 31.8 | n/a |
| BC (ResNet-18) | 10/10 | 0/10 | 8/10 | 3/10 | **52** [37, 67] | 3.2 | 14.3 | 7.8 | 11.6 | 9.2 | 126 |

Table 5 gives the outcome of all 160 main-configuration episodes. Behaviour cloning solves 52% of the held-out episodes (21/40, 95% interval 37% to 67%). The two latent planners solve 10% (DINO-WM, 4/40) and 5% (V-JEPA 2-AC, 2/40), and TD-MPC2 solves none. Every latent-planner success is on Stove; neither planner completes a single pick-and-place, push or drawer episode. The paired tests of Table 6 make the ranking precise. Behaviour cloning beats each world model on the same 40 episodes ($p < 0.001$, exact McNemar), while V-JEPA 2-AC and DINO-WM are statistically indistinguishable ($p = 0.69$): at this sample size the choice between video and image features does not change the closed-loop outcome.

The goal errors locate the failure. On Bowl-Plate, Push-Plate and Drawer the planners end with errors of 16 to 26 cm, essentially the starting errors, so the manipulated object or drawer barely moved. On Stove, where the error is the gripper's

**TABLE 6.** Paired comparisons on the 40 matched episodes. ΔSR is the difference in pooled success (percentage points, first minus second); $p_{\text{McN}}$ is the two-sided exact McNemar test; Δerr is the median paired difference in final goal error (cm) and $p_{\text{W}}$ the Wilcoxon signed-rank test.

| Comparison | ΔSR (pp) | $p_{\text{McN}}$ | Δerr (cm) | $p_{\text{W}}$ |
|---|---|---|---|---|
| V-JEPA 2-AC vs DINO-WM | −5.0 | 0.688 | +0.0 | 0.371 |
| V-JEPA 2-AC vs TD-MPC2 | +5.0 | 0.500 | +0.0 | 0.467 |
| V-JEPA 2-AC vs BC (ResNet-18) | −47.5 | <0.001 | +12.0 | <0.001 |
| DINO-WM vs TD-MPC2 | +10.0 | 0.125 | +0.0 | 0.122 |
| DINO-WM vs BC (ResNet-18) | −42.5 | <0.001 | +13.3 | <0.001 |
| TD-MPC2 vs BC (ResNet-18) | −52.5 | <0.001 | +14.2 | <0.001 |

distance to its goal position, every episode starts 37 cm from the goal; V-JEPA 2-AC ends 34 cm away on average and DINO-WM 20 cm away, while TD-MPC2 drives the arm 73 cm away. Behaviour cloning ends within 3.2 cm on Stove and 7.8 cm on Push-Plate. Its weakest task, Bowl-Plate (0/10), fails at the grasp: in every episode the gripper comes within 3.6 to 7.7 cm of the bowl, yet the bowl never rises more than 3.8 cm, and the failure disappears when the policy also sees the wrist camera (9/10, Table 10). Successful planner episodes take longer than behaviour cloning on the same task (150 and 160 control steps for V-JEPA 2-AC and DINO-WM on Stove against 101 for BC).

The training curves (Fig. 3) rule out under-training as the explanation. Both latent predictors fit their training clips (V-JEPA 2-AC L1 loss 0.270, DINO-WM MSE 0.0172) and reach their lowest held-out loss early (V-JEPA 2-AC 0.365 at epoch 11), after which the held-out loss rises slightly to 0.385: with 13,401 clips, an 11M-parameter predictor saturates what the data can teach. TD-MPC2's held-out latent consistency error grows fivefold over training (from $2.2 \times 10^{-4}$ to $1.1 \times 10^{-3}$) while its training error stays flat at $2.8 \times 10^{-4}$, the signature of a value-equivalent model overfitting 160 successful trajectories.

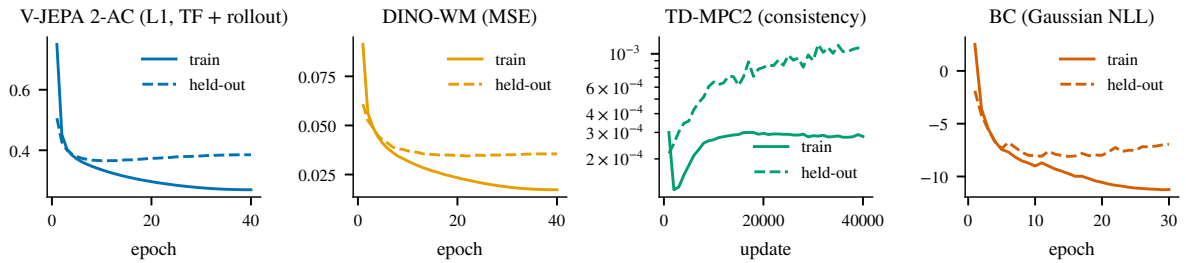



**FIGURE 3.** Training and held-out curves of the four models (seed 0). Held-out curves use the 40 test demonstrations and are logged for monitoring only; no checkpoint is selected on them.

## B. LATENT PREDICTION FIDELITY AND CONTROLLABILITY (RQ2)

**TABLE 7.** Open-loop latent prediction on all held-out windows under the demonstrated actions. Error is the mean L1 (latent world models) or MSE (TD-MPC2) between imagined and encoded latents at horizon $k$; the ratio divides it by the error of predicting no change (copy-last); drift is the mean increase of the error per macro step; the last two columns divide the error obtained with shuffled actions by the error obtained with the true actions at $k = 1$ and $k = 10$ (1.00 means the model ignores its actions).

| | | Error at $k$ | | | | Error / copy-last at $k$ | | | | Drift | Shuffled / true | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Model | Metric | 1 | 3 | 5 | 10 | 1 | 3 | 5 | 10 | per step | $k$=1 | $k$=10 |
| V-JEPA 2-AC | L1 | 0.1872 | 0.2033 | 0.2164 | 0.2381 | 0.93 | 0.79 | 0.75 | 0.74 | 0.0057 | 1.07 | 1.28 |
| DINO-WM | L1 | 0.1263 | 0.1535 | 0.1687 | 0.2020 | 0.77 | 0.62 | 0.58 | 0.57 | 0.0084 | 1.25 | 1.83 |
| TD-MPC2 | MSE | 0.0007 | 0.0015 | 0.0023 | 0.0044 | 0.26 | 0.14 | 0.13 | 0.19 | 0.0004 | 5.13 | 5.95 |

Table 7 and Fig. 4 evaluate the dynamics models open loop on all 618 held-out windows, under the demonstrated actions and exactly as at planning time. Normalised by the error of predicting no change, V-JEPA 2-AC's error is 0.93 at one step and 0.74 at ten steps, and DINO-WM's is 0.77 and 0.57. Both latent models therefore predict better than no-change at every horizon, and their relative advantage grows with $k$ because the scene changes more over longer intervals than their error grows: in absolute terms the error rises by 27% (V-JEPA 2-AC, L1 from 0.187 to 0.238) and 60% (DINO-WM, 0.126 to 0.202) from one to ten steps, while the copy-last error rises by 60% and 116%. Both models also use their actions. Replacing the demonstrated actions by those of another clip raises V-JEPA 2-AC's error by a factor of 1.07 at one step and 1.28 at ten, and DINO-WM's by 1.25 and 1.83; DINO-WM is the more accurate and the more action-sensitive of the two, consistent with, though not established by, its slightly higher closed-loop success (4/40 against 2/40, $p = 0.69$). V-JEPA 2-AC's one-step prediction is nearly action-blind (a 7% effect), so a single macro action barely changes the frame it predicts. TD-MPC2's latent is the most predictable of the three (error 0.26 of copy-last at one step) because its training objective shapes the latent to be predictable, which is no evidence that it encodes the task. The decisive contrast is with closed loop: on the actions the planner executes, the same predictors are

25% to 36% worse than copy-last at one step (Section VI-D), against 7% to 23% better on demonstrated actions. Open-loop accuracy on the data distribution does not carry over to the actions an optimiser selects, which is why the fidelity axis alone overstates the utility of these models for control.

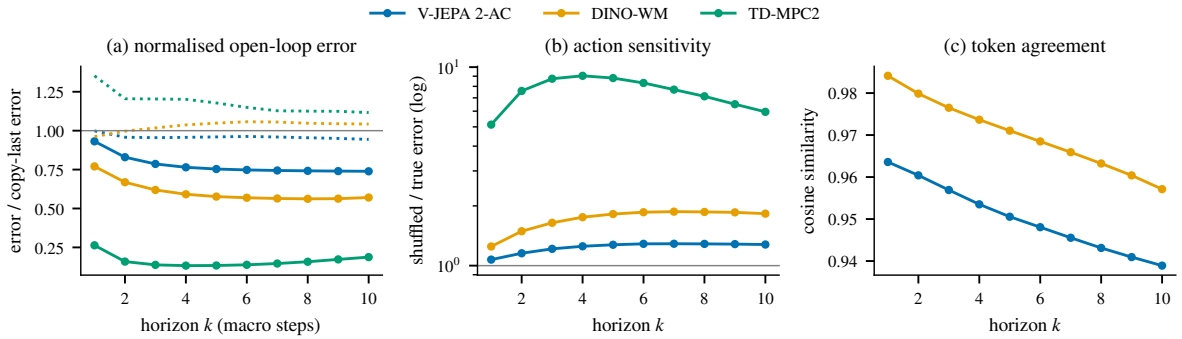



**FIGURE 4.** Open-loop prediction on held-out demonstrations (L1 for the latent world models, MSE in its own latent space for TD-MPC2). (a) Error relative to the copy-last baseline (solid: demonstrated actions; dotted: actions of another clip). (b) Ratio of the shuffled-action error to the true-action error on a log scale; 1 means the prediction ignores the action. (c) Cosine similarity between imagined and encoded token grids.

## C. COMPUTATIONAL PROFILE (RQ3)

**TABLE 8.** Isolated computational profile per decision (one process on an otherwise idle GPU, 30 timed decisions after 3 warm-up decisions). Encode (ms) is the visual encoder on the current frame, plan (ms) the CEM, MPPI or policy forward pass, extract (ms) the transfer of the action to the host. For behaviour cloning the image encoder runs inside the policy forward pass, so its time is counted under plan.

| Model | Frozen | Trained | Encode | Plan | Extract | Total (ms) | Decisions/s | Peak VRAM (GB) |
|---|---|---|---|---|---|---|---|---|
| V-JEPA 2-AC | 303.9M | 11.4M | 38.7 | 472.5 | 0.16 | 511.4 ± 3.1 | 1.96 | 4.00 |
| DINO-WM | 21.6M | 11.3M | 3.9 | 924.8 | 0.18 | 928.8 ± 0.3 | 1.08 | 1.60 |
| TD-MPC2 | 0 | 5.6M | 0.9 | 18.3 | 0.05 | 19.2 ± 0.6 | 52.06 | 0.05 |
| BC (ResNet-18) | 0 | 11.8M | 0.0 | 1.7 | 0.05 | 1.8 ± 0.0 | 556.75 | 0.08 |

Measured in isolation (Table 8), one V-JEPA 2-AC decision takes 511 ms, almost all of it in the CEM, which evaluates $N \times I = 2{,}560$ ten-step rollouts of the predictor; the behaviour-cloning policy takes 2 ms, a factor of 285. DINO-WM is slower than V-JEPA 2-AC per decision despite its smaller encoder, because its sliding three-frame window cannot reuse keys and values, while V-JEPA 2-AC's block-causal predictor can (Section III-A). Peak GPU memory is dominated by the planner's batch of imagined token grids rather than by the encoder: the frozen V-JEPA 2 encoder accounts for 1.2 GB of V-JEPA 2-AC's 4.1 GB peak. Fig. 5 places the four models on the success-latency plane: the direct policy dominates, being both the fastest and the most successful, and no world model reaches the Pareto front.

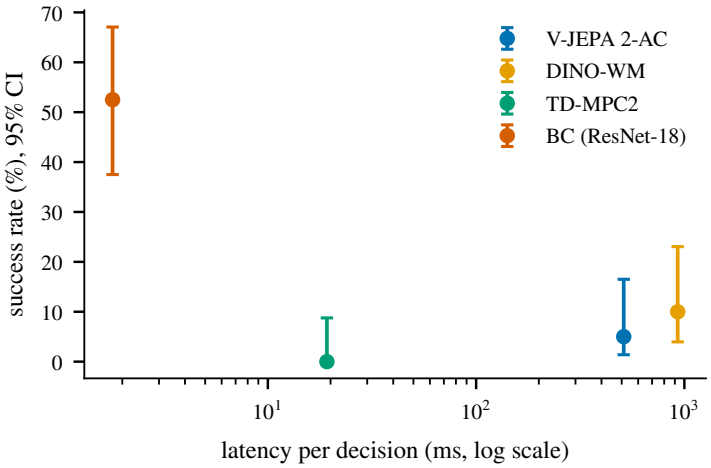



**FIGURE 5.** Success rate (pooled over 40 episodes, Wilson 95% interval) against isolated latency per decision.

### D. WHY THE LATENT PLANNERS SOLVE STOVE AND NOTHING ELSE

Three measurements explain the split between the reach-and-press task and the three contact-rich tasks. First, the planning energy follows the arm, not the object. Along every planner episode we correlated the energy of the observed frame with the gripper's distance to its goal position and, on the three object tasks, with the object's goal error. The median rank correlation with the gripper distance is 0.36 for V-JEPA 2-AC and 0.52 for DINO-WM, and with the object error 0.21 and 0.10; in the remaining object episodes the object never moved, so no correlation is defined (15 and 20 of 30 episodes had any object motion). Matching the arm pose of the goal frame is enough to press the stove button and is not enough to move a bowl, a plate or a drawer. Second, the planners rarely reach the part they must manipulate: 30 of V-JEPA 2-AC's 38 failures and 19 of DINO-WM's 36 never bring the gripper within the contact radius (Section VI-D). Third, the long horizon hurts: shortening it from ten to three macro steps raises V-JEPA 2-AC's Stove success from 2/10 to 9/10 (Section VII), which points to how the planner uses the model (credit assignment over a long horizon) rather than to the representation.

## VI. QUALITATIVE RESULTS AND VISUAL ROLLOUT ANALYSIS

### A. ROLLOUT FILMSTRIPS

Fig. 6 shows eight evenly spaced agent-view frames from successful and failed episodes on Stove and Push-Plate, next to the goal frame the planners were given. The successful planner episodes on Stove move the gripper to the stove and press the button, as behaviour cloning does, but more slowly (215 and 100 control steps in the shown episodes against 125 for behaviour cloning). In the failed Stove episodes shown, the gripper ends beside or below the stove front, 24 to 27 cm from its goal position, without touching the button. The Push-Plate rows show the mechanism of Section V-D directly. Both planners drive the arm toward the region where the arm appears in the goal frame and never touch the plate, which ends about where it started (26 cm from its goal, against 25 cm at the start); behaviour cloning instead approaches the plate from behind and pushes it to the front of the stove. Its failure on Push-Plate reaches the plate but moves it by only about 1 cm.

### B. IMAGINED VERSUS REALISED LATENT TRAJECTORIES

Fig. 7 projects, for every Stove episode, the pooled latent of each observed frame (lines), the pooled latent of the final frame of each plan the CEM selected (crosses) and the goal embedding (stars), by PCA fitted on the realised latents and by t-SNE [41] fitted on all three. Two observations hold for both planners. First, the imagined plan ends do not lie on the manifold of real frames: in t-SNE they form clusters of their own, and their median distance to the nearest real latent is 1.7 (V-JEPA 2-AC) and 1.8 (DINO-WM) times the median nearest-neighbour distance among real latents. Second, they collapse toward the centre of the data: their spread around their own mean is 0.41 and 0.70 of the spread of the real latents. A regression model trained with an L1 or L2 loss on uncertain futures predicts something close to their average, and averaging embeddings of different arm and object configurations produces an embedding of no real configuration. The planner then compares the goal with these averaged futures, so its choice is driven by whatever features survive averaging, mainly the coarse position of the arm.

### C. ENERGY LANDSCAPES AND OPTIMISER BEHAVIOUR

Fig. 8 slices the planning problem at the first decision of a held-out Stove episode and a held-out Push-Plate episode. The first macro action's $(\Delta x, \Delta y)$ command is varied over an $11 \times 11$ grid while the rest of the selected plan is kept; the top row shows the energy the world model predicts, the bottom row the true goal error after executing the same plan for ten macro steps in the simulator, and the white path is the CEM mean of the first action across iterations. At the first Stove decision, V-JEPA 2-AC's energy varies by only 5.4% of its mean across the 121 first actions, and its rank correlation with the executed outcome is 0.16: the plan the energy prefers ends 28.1 cm from the goal, while the best plan on the grid ends at 25.2 cm. With a three-step horizon the landscape of the same state is twice as steep (10.7%) and clearly aligned with the outcome ($\rho_s = 0.57$), which is the horizon ablation of Section VII-A seen at the level of a single decision. DINO-WM's Stove landscape is steeper (35.2%) and moderately aligned ($\rho_s = 0.36$), but its minimum still selects a plan ending 11.7 cm from the goal against 2.8 cm for the best plan on the grid. On Push-Plate no perturbation of the first action moves the plate within the ten-step plan: the true goal error is 26.1 cm at all 121 grid points, so a rank correlation is undefined. The predicted energy nevertheless varies across the grid, by 1.1% for V-JEPA 2-AC and 67.4% for DINO-WM, and the CEM descends it. Because the plate does not move, those differences can only reflect the arm: the optimiser minimises a quantity that does not depend on the object it has to move, which is the energy-attribution result of Section V-D seen from the planner's side.

### D. FAILURE TAXONOMY

We classify every episode by its physical outcome and, for the latent planners, by two diagnostics computed along the episode (Fig. 9). An episode *never reached the part* if the gripper never came within the contact radius of the body it must manipulate (the button, bowl, plate or drawer), with the radius set per task to 1.5 times the median gripper-to-part distance at the moment of success over all successful episodes (Stove 4.2 cm, Bowl-Plate 8.5 cm, Push-Plate 10.9 cm, Drawer 24.1 cm). It made *contact without progress* if it reached the part but reduced the goal error by less than 25%, and *partial progress* otherwise. For the planners, *goal-embedding ambiguity* is flagged when the planning energy of the observed frames does not rank them by physical goal error (Spearman $\rho_s < 0.3$ along the episode), and *compounding open-loop drift* when the predicted next latent under the executed action is, at the median decision, further from the observed next latent than the current latent is (one-step error over copy-last error above 1). These three regimes correspond to the contact-discontinuity, goal-ambiguity and drift failure modes anticipated in Section I-B. The outcome categories in Fig. 9a separate the models sharply. Of their failures, 30 for V-




**FIGURE 6.** Executed rollouts (8 evenly spaced frames, $t$ in control steps) for V-JEPA 2-AC, DINO-WM and behaviour cloning on Stove and Push-Plate, with the goal frame on the right (blue frame: success, orange: failure). The label gives the final goal error.

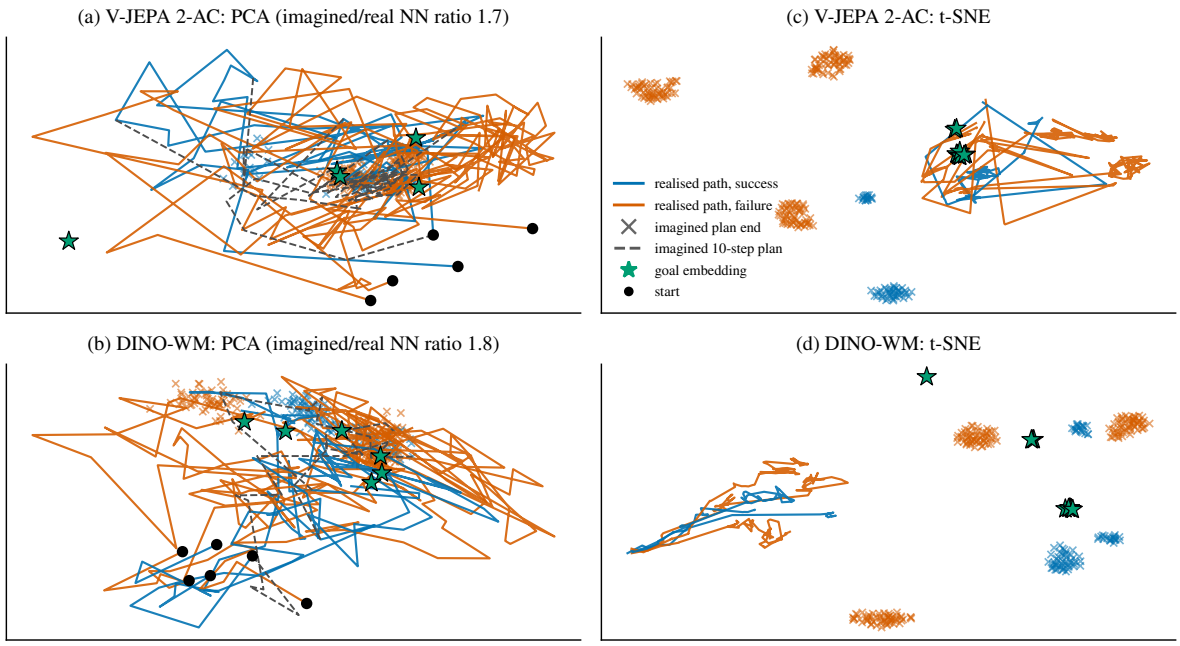



**FIGURE 7.** Latent manifolds on Stove. (a, b) PCA of realised paths (lines; blue success, orange failure), imagined final frames of every selected plan (crosses), imagined 10-step plans from every eighth decision of one episode (dashed) and goal embeddings (stars). (c, d) t-SNE of the same points. The title gives the median distance of imagined plan ends to their nearest realised latent divided by the median nearest-neighbour distance among realised latents.

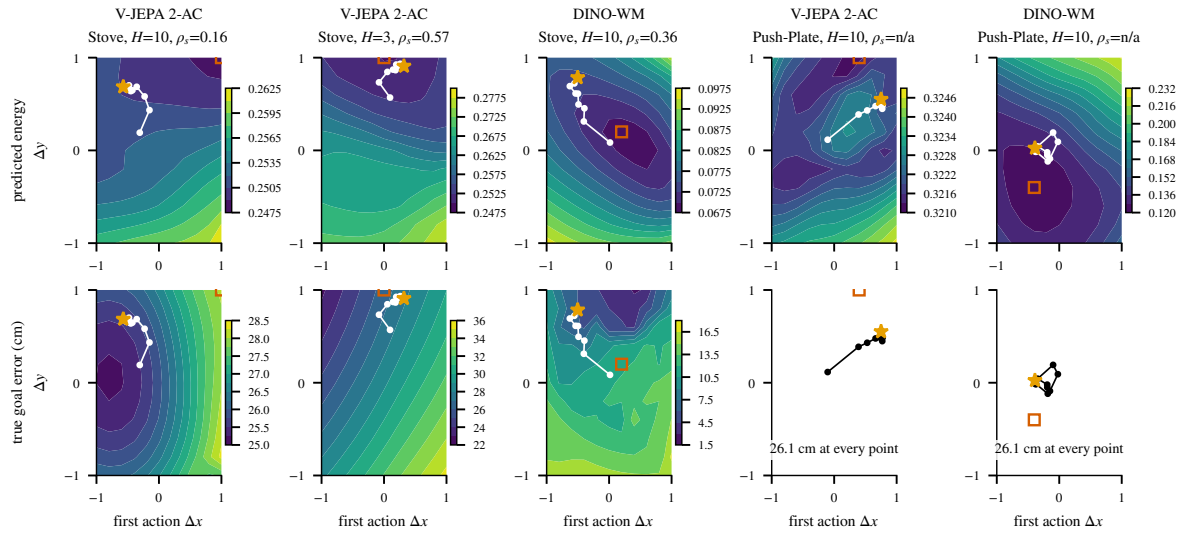



**FIGURE 8.** Predicted energy (top) and true goal error after executing the plan (bottom) over the first action's $(\Delta x, \Delta y)$ command, at the first decision of held-out episodes. White: CEM mean of the first action per iteration (orange star: final). Square: grid minimum. $\rho_S$ is the Spearman rank correlation between the two surfaces over the 121 grid points (n/a where the executed outcome is the same at every point).

JEPA 2-AC, 19 for DINO-WM and all 40 for TD-MPC2 never reached the part, while behaviour cloning reached it in every one of its failures and failed at the contact (18 episodes of contact without progress, mostly the Bowl-Plate grasp). The world models therefore fail before the contact problem even arises, behaviour cloning fails at it. The latent diagnostics in Fig. 9b show the other two regimes. *Goal-embedding ambiguity* is common: in 25 of 38 failed V-JEPA 2-AC episodes and 22 of 36 failed DINO-WM episodes, the planning energy of the observed frames ranks them by physical goal error with $\rho_S < 0.3$ (median $\rho_S$ 0.21 and 0.22, against 0.57 in DINO-WM's successful episodes). *Goal hallucination* accompanies it: in failed episodes the planners believe each plan reduces the energy by 23.8% and 55.5% on median, more than in successful ones (6.9% and 31.4%), so the largest promised progress comes in the episodes that make none. *Open-loop drift* is not discriminative in closed loop: in every planner episode, successful or not, the predicted next latent under the executed action is further from the observed next latent than the current latent is (median ratio 1.28 and 1.25 in failures, 1.36 and 1.26 in successes), because the actions the planner executes lie outside the demonstrations (Section VII-A); its growth with the horizon is measured open loop in Table 7.

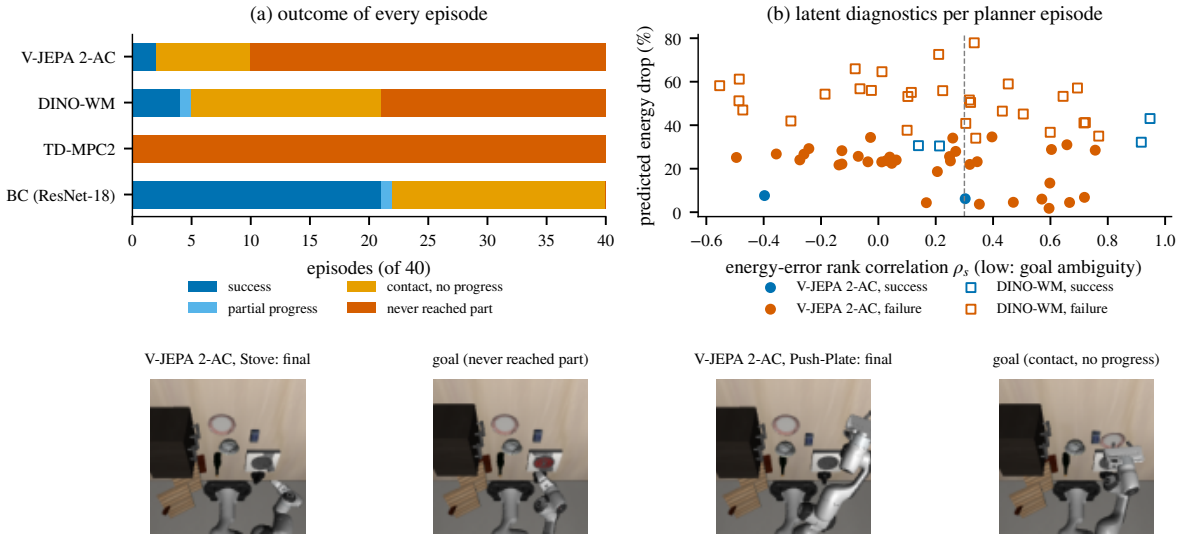



**FIGURE 9.** Failure taxonomy. (a) Outcome of all 40 episodes per model. (b) Per-episode latent diagnostics of the two planners: rank correlation between the planning energy of the observed frames and their true goal error (left of the dashed line: goal-embedding ambiguity) against the median energy drop the planner predicted for its selected plans (high values in failed episodes: goal hallucination). (c) Final frame and goal of representative V-JEPA 2-AC failures.

## VII. ABLATION STUDIES AND SENSITIVITY ANALYSIS

### A. PLANNER DESIGN (RQ4)

**TABLE 9.** Planner ablations for V-JEPA 2-AC on Stove (10 episodes each, seed 0). One setting is changed at a time from the main configuration. Min. error is the closest the task-relevant part came to its goal during the episode; plan time is the mean CEM time per decision during the run (shared GPU).

| Factor | Setting | Success [95% CI] | Final error (cm) | Min. error (cm) | Steps to success | Plan time (s) |
|---|---|---|---|---|---|---|
| Main | $H$=10, $N$=256, $\kappa$=0.05, L1 | 2/10 [6, 51] | 34.0 | 10.1 | 150 | 8.3 |
| Horizon | $H$=3 | 9/10 [60, 98] | 10.4 | 5.0 | 115 | 1.6 |
| Horizon | $H$=5 | 5/10 [24, 76] | 19.4 | 5.1 | 94 | 2.7 |
| Horizon | $H$=15 | 3/10 [11, 60] | 44.9 | 6.9 | 149 | 9.0 |
| Samples | $N$=32 | 1/10 [2, 40] | 54.6 | 9.5 | 192 | 1.8 |
| Samples | $N$=64 | 2/10 [6, 51] | 37.2 | 10.6 | 188 | 2.3 |
| Samples | $N$=128 | 1/10 [2, 40] | 32.2 | 8.1 | 193 | 3.4 |
| Samples | $N$=512 | 2/10 [6, 51] | 35.8 | 10.7 | 146 | 10.0 |
| Elites | $\kappa$=0.1 | 2/10 [6, 51] | 30.6 | 10.2 | 238 | 5.4 |
| Elites | $\kappa$=0.2 | 1/10 [2, 40] | 33.3 | 15.9 | 103 | 4.9 |
| Distance | cosine | 1/10 [2, 40] | 43.1 | 11.1 | 137 | 4.7 |
| Distance | Euclidean $L_2$ | 0/10 [0, 28] | 44.3 | 11.0 | n/a | 4.5 |
| Distance | Mahalanobis (diag.) | 1/10 [2, 40] | 32.2 | 10.4 | 275 | 4.5 |

All planner ablations use V-JEPA 2-AC on Stove, the only task on which either latent planner succeeds in its main configuration, and change one setting at a time (Table 9).

*1) Planning horizon.* The horizon is the only planner setting that changes the outcome. Shortening the plan from ten to five and to three macro steps raises Stove success from 2/10 to 5/10 and 9/10 (Wilson interval 60% to 98%; paired exact McNemar against the main configuration on the same ten episodes, $p = 0.039$), lowers the mean final gripper error from 34.0 to 10.4 cm

and cuts planning time per decision from 8.3 to 1.6 s. Lengthening it to 15 steps gives 3/10 with the largest final error, 44.9 cm. Compounding error alone does not explain the size of the effect, since the absolute open-loop error grows by only 27% from one to ten steps (Section V-B). The executed actions point to credit assignment. The energy scores only the final imagined frame, and only the first action of the plan is executed; with ten steps the first action has a small share of the influence on that frame, so the CEM leaves it poorly determined and its mean shrinks toward zero. Measured on the recorded Stove episodes, the executed translation command averages 0.60 of its maximum with $H = 10$ and 0.99 with $H = 3$, and the gripper approaches its goal at 1.04 and 1.23 cm per decision over the first 20 decisions. We read this as the mechanism, an inference supported by these measurements rather than proven by them. The finding also agrees with the released V-JEPA 2-AC planner, which looks only one or two steps ahead [15]. The main configuration's ten-step horizon, fixed before any experiment, was therefore the wrong choice for this model, and Section VII-A5 tests the shorter horizon on the other tasks.

*2) Sample budget and elite fraction.* Neither the sample budget nor the elite fraction moves the outcome beyond sampling noise: $N$ from 32 to 512 gives 1 or 2 successes out of 10, and elite fractions of 5%, 10% and 20% give 2, 2 and 1. Planning time grows roughly linearly with $N$, from 1.8 s at $N = 32$ to 10.0 s at $N = 512$, so beyond 32 samples additional compute buys nothing. This is what one expects when the limitation is the predicted energy rather than the search: Section VI-C shows that at $H = 10$ the energy surface is shallow and only weakly aligned with the executed outcome, so more samples find its minimum more reliably without finding a better plan.

*3) Latent distance.* Replacing the release's L1 energy by the cosine, Euclidean or diagonal Mahalanobis distance gives 1, 0 and 1 successes against 2 for L1, and the closest approach to the goal is 10 to 11 cm under every distance. The distances only reweight channels of the same imagined frames, so they cannot repair an imagined frame that is wrong; L1, the release's choice, is at least as good as the alternatives.

*4) Action prior.* The planners' executed rotation commands were six times larger on average than the demonstrated ones (mean absolute value 0.30 against 0.05) and their gripper commands averaged 0.26 in magnitude, where the demonstrations average 0.98. To test whether these out-of-distribution actions cause the failures, we replaced the CEM's initial spread of 1 in every dimension by the per-dimension standard deviation of the demonstrated macro actions (0.36 to 0.42 for translation, 0.05 to 0.11 for rotation, 0.77 for the gripper) and re-ran both planners on all four tasks. The prior removed the out-of-distribution commands but did not help: V-JEPA 2-AC solved 0/40 (main configuration 2/40, $p = 0.500$) and DINO-WM 0/40 (main 4/40, $p = 0.125$). With DINO-WM it brought the gripper closer to its goal position on Stove (final error 9.4 cm against 20.3 cm) without pressing the button. Out-of-distribution actions are therefore a symptom rather than the cause of failure. A plausible reading, which we did not test further, is that the energy is nearly flat in the rotation and gripper dimensions, so the CEM leaves them at whatever spread it starts from, and that the narrower prior also slows exploration in the translation dimensions the energy does resolve.

*5) Short horizon on all tasks (post hoc).* Because the horizon was chosen on the ten Stove episodes, we re-ran both planners with $H = 3$ on all four tasks and treat Bowl-Plate, Push-Plate and Drawer, which played no part in the choice, as an out-of-selection test. V-JEPA 2-AC with $H = 3$ solves 9/10 Stove episodes and 0, 0 and 0 of the Bowl-Plate, Push-Plate and Drawer episodes (9/40 pooled, against 2/40 at $H = 10$, $p = 0.039$); DINO-WM solves 5/10 Stove episodes and 0, 0 and 0 of the others (5/40, against 4/40, $p = 1.000$). The shorter horizon turns V-JEPA 2-AC into a reliable reach-and-press planner and leaves the three contact-rich tasks unsolved, which separates the two failure causes cleanly: credit assignment over a long horizon explains the Stove failures, missing contact knowledge explains the rest. Even at its best horizon, V-JEPA 2-AC remains below behaviour cloning ($p < 0.001$).

## B. FROZEN VERSUS FINE-TUNED REPRESENTATIONS

**TABLE 10.** Representation ablation for behaviour cloning: frozen pretrained features versus end-to-end learning. Held-out MSE is the action error on the 4,993 test frames; successes are out of 10 per task (Stove, Bowl-Plate, Push-Plate, Drawer).

| Policy | Trained (M) | Held-out MSE | Stove | Bowl | Push | Drawer | Pooled [95% CI] | Error (cm) |
|---|---|---|---|---|---|---|---|---|
| BC (ResNet-18) | 11.8 | 0.0363 | 10 | 0 | 8 | 3 | 21/40 [37, 67] | 9.2 |
| BC + wrist view | 23.2 | 0.0207 | 10 | 9 | 10 | 9 | 38/40 [83, 99] | 3.2 |
| BC, frozen DINOv2 | 0.8 | 0.0252 | 10 | 9 | 9 | 7 | 35/40 [74, 95] | 3.8 |
| BC, frozen V-JEPA 2 | 1.0 | 0.0245 | 9 | 7 | 9 | 0 | 25/40 [47, 76] | 7.5 |
| BC, fine-tuned DINOv2 | 22.1 | 0.0206 | 10 | 10 | 10 | 5 | 35/40 [74, 95] | 4.3 |

Table 10 asks the representation question with the planner removed: the same frozen encoders that the latent world models plan in are given to a behaviour-cloning head. Frozen self-supervised features are a better substrate for a policy than end-to-end learning from 160 demonstrations. Attention pooling over frozen DINOv2 tokens (0.8M trained parameters) solves 35/40, against 21/40 for the end-to-end ResNet-18 with 11.8M trained parameters ($p = 0.001$), and fine-tuning DINOv2 end to end reaches the same 35/40. Frozen V-JEPA 2 tokens reach 25/40 ($p = 0.388$ against ResNet-18) but fail every Drawer episode, where frozen DINOv2 solves 7/10, and are significantly worse than frozen DINOv2 as a policy input ($p = 0.013$). The largest single gain is the wrist camera (38/40, $p < 0.001$), which removes the Bowl-Plate grasp failure (0/10 to 9/10). The central comparison of the study follows. The same frozen features that support 25 to 35 successful episodes as a policy input support 2 to 4 as a planning space, so the representation is not what limits the latent world models; the action-conditioned dynamics learned from 40 demonstrations per task, and the planner's dependence on it, are.

## VIII. DISCUSSION, LIMITATIONS AND FUTURE DIRECTIONS

### A. WHY LATENT WORLD MODELS WITHOUT CONTACT GROUNDING STRUGGLE

The four experiments point to one mechanism. A latent world model trained on 40 demonstrations per task learns the visually dominant regularity of the data, the motion of the arm, and its planning energy inherits that bias: along the episodes the energy follows the gripper's distance to its goal pose ($\rho_s$ 0.36 and 0.52) far more than the object's distance to its goal (0.21 and 0.10). Reaching and pressing a button is an arm-pose problem, and both planners solve it. Moving a bowl, a plate or a drawer requires the model to predict what happens at a contact, which is a discontinuous, low-pixel-count event that 40 demonstrations show only a few dozen times, and neither planner learns it: the objects never moved in most episodes, and the majority of failures never brought the gripper within reach of the part (Fig. 9). The imagined futures also regress toward an average embedding (Fig. 7): the planner's selected plan ends lie 1.7 and 1.8 times further from the nearest real frame than real frames lie from each other, and they occupy only 0.41 and 0.70 of the realised latents' spread. An optimiser that searches against such predictions finds plans that look like progress in latent space and are not; in failed episodes the planners predicted energy drops of 23.8% and 55.5% per decision, larger than in their successful episodes (6.9% and 31.4%), which is the goal-hallucination signature of model exploitation. The horizon result fits the same picture. Short plans keep the comparison with the goal inside the range where the predictor is accurate, and on Stove they convert a 2/10 planner into a 9/10 one; they cannot create contact knowledge that the model does not have, which is what the post-hoc test on the other tasks examines (Section VII-A5).

These findings separate the three axes that Assignment 1 asked to keep apart. *Representation*: frozen self-supervised features are a good substrate, better than end-to-end learning for a policy (35/40 against 21/40). *Dynamics fidelity and controllability*: the action-conditioned predictors on top of them are where the transfer fails at this data scale. *Control utility*: test-time optimisation over those predictors costs two orders of magnitude more latency than a policy and, in the main configuration, delivers between a tenth and a fifth of its success.

### B. IMPLICATIONS FOR PHYSICAL FOUNDATION MODELS AND A ROADMAP

Three implications follow for the use of world models as foundation models for manipulation, including the 5-D and 6-D policies that must control position, orientation and grasp jointly. First, a pretrained visual representation and a world model built on it are separable assets with different readiness: the representation transferred (Table 10), the 40-demonstration dynamics model did not. Claims that a world model is a transferable substrate should therefore be tested on the dynamics, with the action-shuffle and energy-versus-outcome diagnostics used here, rather than on representation probes alone. Second, the failure is concentrated in the six dimensions that matter for 6-D manipulation: the planners commanded rotations six times larger on average than the demonstrations (0.30 against 0.05) and gripper commands of magnitude 0.26 where the demonstrations command 0.98, so a planner that searches the full pose space needs either data that covers it or a prior that restricts it (Section VII). Third, the cost is structural: test-time optimisation multiplies the predictor's cost by samples, iterations and horizon, which on this hardware put a decision two orders of magnitude above a policy forward pass (Table 8).

A route toward hybrid world models follows from where the latent models failed. (i) *Geometry in the state.* Contact-rich failures occurred where the $8 \times 8$ grid of frozen features cannot resolve a 4 cm button or a drawer handle; replacing or augmenting patch tokens with depth tokens or 3-D Gaussian primitives [40] anchored to object surfaces would give the predictor a representation in which contact and articulation are explicit. (ii) *Action-covering data.* V-JEPA 2-AC was trained on tens of hours of diverse DROID interaction [15, 30]; at the scale of 40 expert demonstrations per task, perturbed or play data that spans the action space is likely worth more than predictor capacity, a conjecture this study did not test. (iii) *Policy-guided search.* TD-MPC2's use of a policy prior inside MPPI [4] suggests seeding latent CEM with a behaviour-cloning proposal, so that the world model ranks plausible actions instead of searching an unconstrained 70-dimensional space. (iv) *Evaluation by executability.* As RoboWM-Bench and GigaWorld-1 argue [19, 20], the planner's energy must be validated against executed outcomes; the energy-versus-outcome correlation of Fig. 8 is a cheap instance of that test and should accompany any latent-planning result.

### C. LIMITATIONS AND THREATS TO VALIDITY

- *Statistical power.* All results use one seed and 10 episodes per task, as specified for this study. A per-task difference of two successes is within sampling noise (a 2/10 result has a Wilson interval of 6% to 51%), so we draw conclusions from pooled, paired tests on 40 episodes and treat per-task contrasts as indicative. Training seeds were not varied, so the variance due to initialisation is unmeasured.
- *Reduced scale.* Both latent predictors have 11M parameters on an $8 \times 8$ grid of frozen ViT-L or ViT-S features; the released V-JEPA 2-AC uses a 300M-parameter predictor on $16 \times 16$ ViT-g tokens trained on DROID [30]. Our results bound what the *recipe* achieves with 40 demonstrations per task at this scale; they do not measure the released checkpoint, which acts in a different embodiment and action space.
- *Data regime.* The demonstrations are near-expert human trajectories. Planning queries the model on actions far from the data, which a model trained only on expert data has never seen; this favours the policy baseline by construction and is part of what the comparison measures.
- *Offline TD-MPC2.* TD-MPC2 is designed for online learning with dense or shaped rewards. Trained offline with a sparse success reward on 160 successful trajectories, its value function sees no failures, so our numbers characterise this regime, not TD-MPC2 in its intended setting.
- *Goal specification.* The latent planners receive the final frame of a successful demonstration from the same initial state, an optimistic goal that also fixes the final arm pose. BC and TD-MPC2 receive only the task identity.

- *Simulation only.* LIBERO renders a fixed camera with clean lighting; nothing here measures robustness to real sensing, which Assignment 1 identified as the open problem for this family [18, 19].

### D. CONCLUSION

Under one protocol, with the data, interface, episodes and seed held fixed, a direct policy outperformed every world model on LIBERO-Goal: behaviour cloning solved 52% of the held-out episodes (95% with a wrist camera), the two latent planners 10% and 5% in their main configuration, and TD-MPC2 none. The answer this gives to the question of Assignment 1 is specific. A pretrained self-supervised video or image representation is a transferable substrate for control: frozen DINOv2 features gave a policy 88% success with 0.8M trained parameters. An action-conditioned latent world model trained on 40 demonstrations per task on top of that representation is not yet one: it predicts well on demonstrated actions and badly on the actions a planner chooses, its energy tracks the arm rather than the object, and planning with it solves the reach-and-press task (90% with a three-step horizon) and none of the contact-rich ones. The measurements that established this (open-loop error under true and shuffled actions, energy-versus-outcome correlation, energy attribution to arm and object, and the split of failures into approach, contact, ambiguity and hallucination) are inexpensive and should accompany any claim that a world model transfers to control.

## IX. REPRODUCIBILITY AND COMPLIANCE

The repository contains the complete source code in `src/` and `scripts/`, one configuration file per trained model in `configs/model/`, unit tests for every model and for the planner plus a simulator integration test (`tests/`, 13 tests), training histories and evaluation logs (`results/logs/`), per-episode evaluation records (`results/eval/`), aggregated tables (`results/tables/`) and a README with the commands that regenerate every number in this report. The LIBERO demonstrations are not redistributed; the README gives the official download command, and `scripts/build_libero_dataset.py` regenerates the processed dataset deterministically. Pretrained weights are downloaded from their official sources on first use. Table 11 acknowledges every external resource, and Table 12 maps each requirement of the assignment to the place where it is met. AI-based coding assistance was used for implementation, debugging and documentation; every component was read, tested (equivalence tests for the cached predictor, convergence tests for the planner, shape and gradient tests for every model) and verified against the released code where one exists.

**TABLE 11.** External code, weights and data.

| Resource | Use in this work | Licence |
|---|---|---|
| `facebookresearch/vjepa2` [15] | encoder and AC-predictor modules vendored unchanged in `src/third_party/vjepa2`; `vitl.pt` weights; CEM update rule | MIT |
| DINOv2 ViT-S/14 [22] via `timm` | frozen encoder of DINO-WM and of two BC variants | Apache-2.0 |
| `gaoyuezhou/dino_wm` [8] | reference for the predictor and planner configuration (re-implemented) | MIT |
| `nicklashansen/tdmpc2` [4] | reference for TD-MPC2 (re-implemented) | MIT |
| LIBERO [21], robosuite [26], MuJoCo [27] | simulator, BDDL tasks, demonstrations, physics and rendering | MIT, MIT, Apache-2.0 |
| ResNet-18 [36] (`timm/resnet18.a1_in1k`) | BC backbone initialisation | Apache-2.0 |

**TABLE 12.** Where each requirement of Assignment 2 is satisfied.

| Requirement | Where it is met |
|---|---|
| Setting 3: 3 to 4 recent models in one simulator | three recently proposed models (V-JEPA 2-AC 2025, DINO-WM 2025, TD-MPC2 2024) and a behaviour-cloning reference, all in LIBERO, Sections III and IV |
| Models related to Assignment 1 | three of the four come from the Assignment 1 corpus ([15, 8, 4]); Section II, Table 1 |
| Consistent experimental settings | same 160 demonstrations, interface, episodes, horizon limit and seed for all models, Sections IV-B and IV-D |
| Simulator, data source, preprocessing, train/validation/test split | Sections IV-A and IV-B, Tables 2 and 3 |
| Input and output structure, input size, batch size, epochs, optimiser, learning rate, loss, augmentation, pretrained weights | Table 4, Section III |
| Hardware and software environment | Section IV-E |
| Suitable metrics (success, reward, convergence, task-specific) | success, return, goal error, steps to success, latency, memory, prediction error, training convergence, Section IV-D, Fig. 3 |
| Comparative evaluation with tables, figures, curves, qualitative outputs | Tables 5 to 10, Figs. 3 to 9 |
| Training and testing logs | `results/logs/`, Fig. 3 |
| Code, configuration files, README, data links | repository layout in the README; `configs/`; LIBERO download command |
| References and external repositories | reference list; Table 11 |
| Explainable implementation | equations and design choices in Section III; tests in `tests/` |

## REFERENCES


[1] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel, "DayDreamer: World models for physical robot learning," in *Proc. 6th Conf. Robot Learning (CoRL)*, PMLR, vol. 205, 2022, pp. 2226-2240. arXiv:2206.14176.

[2] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. B. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2302.00111.

[3] S. Yang, Y. Du, S. K. S. Ghasemipour, J. Tompson, L. P. Kaelbling, D. Schuurmans, and P. Abbeel, "Learning interactive real-world simulators," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.06114.

[4] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.16828.

[5] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong, "Unleashing large-scale video generative pre-training for visual robot manipulation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2312.13139.

[6] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, *et al.*, "Genie: Generative interactive environments," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR, vol. 235, 2024. arXiv:2402.15391.

[7] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long, "iVideoGPT: Interactive VideoGPTs are scalable world models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. arXiv:2405.15223.

[8] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025. arXiv:2411.04983. Code: https://github.com/gaoyuezhou/dino_wm.

[9] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, "Navigation world models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 15791-15801. arXiv:2412.03572.

[10] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen, "Video Prediction Policy: A generalist robot policy with predictive visual representations," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, PMLR, vol. 267, 2025. arXiv:2412.14803.

[11] NVIDIA, N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, *et al.*, "Cosmos world foundation model platform for physical AI," Technical report, 2025. arXiv:2501.03575.

[12] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, no. 8059, pp. 647-653, 2025. doi:10.1038/s41586-025-08744-2.

[13] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta, "Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets," in *Proc. Robotics: Science and Systems (RSS)*, 2025. arXiv:2504.02792.

[14] J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, *et al.*, "DreamGen: Unlocking generalization in robot learning through video world models," in *Proc. 9th Conf. Robot Learning (CoRL)*, PMLR, vol. 305, 2025, pp. 5170-5194. arXiv:2505.12705.

[15] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, *et al.*, Y. LeCun, M. Rabbat, and N. Ballas, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," Technical report, 2025. arXiv:2506.09985. Code and weights: https://github.com/facebookresearch/vjepa2.

[16] Y. Liao, P. Zhou, S. Huang, D. Yang, S. Chen, Y. Jiang, Y. Hu, J. Cai, *et al.*, "Genie Envisioner: A unified world foundation platform for robotic manipulation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026. arXiv:2508.05635.

[17] Y. Guo, L. X. Shi, J. Chen, and C. Finn, "Ctrl-World: A controllable generative world model for robot manipulation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026. arXiv:2510.10125.

[18] Z. Zhang, Z. Li, B. Rahmati, R. H. Yang, Y. Ma, A. Rasouli, *et al.*, "Do world action models generalize better than VLAs? A robustness study," preprint, 2026. arXiv:2603.22078.

[19] F. Jiang, Y. Chen, K. Xu, Y. Liu, H. Wang, Z. Shen, J. Lu, S. Huang, Y. Wang, C. Xie, and R. Wu, "RoboWM-Bench: A benchmark for evaluating world models in robotic manipulation," preprint, 2026. arXiv:2604.19092.

[20] GigaWorld Team, A. Ma, B. Wang, B. Li, C. Ni, G. Li, G. Huang, G. Zhao, *et al.*, "GigaWorld-1: A roadmap to build world models for robot policy evaluation," preprint, 2026. arXiv:2607.02642.

[21] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023. arXiv:2306.03310. Code and data: https://github.com/Lifelong-Robot-Learning/LIBERO.

[22] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024. arXiv:2304.07193.

[23] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "Vision transformers need registers," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2309.16588.

[24] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 15619-15629. arXiv:2301.08243.

[25] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas, "Revisiting feature prediction for learning visual representations from video," *Transactions on Machine Learning Research*, 2024. arXiv:2404.08471.

[26] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, "robosuite: A modular simulation framework and benchmark for robot learning," preprint, 2020. arXiv:2009.12293.

[27] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026-5033. doi:10.1109/IROS.2012.6386109.

[28] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. 5th Conf. Robot Learning (CoRL)*, 2021. arXiv:2108.03298.

[29] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "OpenVLA: An open-source vision-language-action model," in *Proc. 8th Conf. Robot Learning (CoRL)*, 2024. arXiv:2406.09246.

[30] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," in *Proc. Robotics: Science and Systems (RSS)*, 2024. arXiv:2403.12945.

[31] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, "A tutorial on the cross-entropy method," *Annals of Operations Research*, vol. 134, no. 1, pp. 19-67, 2005. doi:10.1007/s10479-005-5724-z.

[32] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, and G. Martius, "Sample-efficient cross-entropy method for real-time planning," in *Proc. 4th Conf. Robot Learning (CoRL)*, PMLR, vol. 155, 2020, pp. 1049-1065. arXiv:2008.06389.

[33] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2017, pp. 1714-1721. doi:10.1109/ICRA.2017.7989202.

[34] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 97, 2019. arXiv:1811.04551.

[35] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine, "Visual Foresight: Model-based deep reinforcement learning for vision-based robotic control," preprint, 2018. arXiv:1812.00568.

[36] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770-778. doi:10.1109/CVPR.2016.90.

[37] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021. arXiv:2010.11929.

[38] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, art. 127063, 2024. doi:10.1016/j.neucom.2023.127063.

[39] I. Kostrikov, D. Yarats, and R. Fergus, "Image augmentation is all you need: Regularizing deep reinforcement learning from pixels," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021. arXiv:2004.13649.

[40] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1-14, 2023. doi:10.1145/3592433.

[41] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579-2605, 2008.

[42] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209-212, 1927. doi:10.1080/01621459.1927.10502953.

[43] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153-157, 1947. doi:10.1007/BF02295996.